

Product Requirements Document

TakboHub

Running Events Management Platform

Field	Value
Version	2.0
Date	January 2026
Location	Dumaguete City, Negros Oriental
Status	Technical Specification Updated

Table of Contents

Executive Summary

The Pitch

Running events in the Philippines are booming—but organizers are still stuck with Google Forms, manual GCash reconciliation, and spreadsheet chaos. TakboHub is a purpose-built platform that lets race organizers create events, collect payments, and manage participants in one place—while giving runners a seamless registration experience with local payment options they already use.

The Philippine running community has experienced explosive growth, with fun runs, trail races, and marathons happening every weekend across the country. Despite this growth, event organizers lack purpose-built tools, relying on fragmented solutions that create operational headaches and poor runner experiences.

TakboHub addresses this gap by providing an end-to-end event management platform designed specifically for the Philippine running market, with native support for local payment methods and mobile-first design.

Problem Statement

For Organizers

- Registration management via Google Forms is manual and error-prone
- Payment collection requires posting GCash/Maya numbers and manually matching screenshot receipts to registrations
- Participant data lives in multiple spreadsheets with no single source of truth
- Communication with runners is fragmented across Facebook, Viber, and email
- Race kit distribution and bib assignment is a logistical nightmare
- No easy way to publish and share results

For Runners

- Discovering upcoming events requires following multiple Facebook pages and groups
- Registration experiences vary wildly in quality and reliability
- Payment confirmation is often delayed or unclear
- No centralized place to track personal race history and PRs

User Roles & Permissions

TakboHub supports multiple user roles to accommodate different stakeholders and workflows within an organization.

Role Definitions

Role	Description	Primary Use Case
Super Admin	Platform owner/operator	Manage all organizations, support, payouts, platform settings
Organizer (Owner)	Event creator and primary manager	Full control over events, staff, payments, and settings
Registration Staff	Helpers with limited access	Confirm payments, add walk-ins, check-in runners on race day
Runner	Event participant	Register for events, view own registrations, optional account

Permission Matrix

Action	Super Admin	Organizer	Staff	Runner
Create events	All	Own org	—	—
Edit event details	All	Own org	—	—
View registrations	All	Own org	Assigned event	Own only
Confirm payments	✓	✓	✓	—
Add walk-in registrations	✓	✓	✓	—
Export data	✓	✓	Limited	—
Manage staff	✓	✓	—	—
View financials/payouts	✓	✓	—	—

MVP Role Scope

- **Phase 1 (MVP):** Super Admin, Organizer, Runner
- **Phase 1.5:** Add Registration Staff role
- **Phase 2:** Walk-in registration mode, race day check-in interface

Payment Methods & Workflows

TakboHub supports both online and offline payment methods to accommodate the diverse payment preferences in the Philippine market.

Supported Payment Methods

Method	Provider	Confirmation	Fees
QR Ph	PayMongo	Automatic	1.5% per transaction (lowest)
GCash	PayMongo	Automatic	2.5% per transaction
Maya	PayMongo	Automatic	2.5% per transaction
GrabPay	PayMongo	Automatic	~2.5% per transaction
Credit/Debit Card	PayMongo	Automatic	3.5% + ₱15 per transaction
Bank Transfer	Manual	Manual (upload proof)	None (organizer's bank)
Cash	Manual	Manual (staff confirms)	None
Free/Comped	N/A	Automatic	None

Payment Status Flow

All registrations follow a consistent status flow regardless of payment method:

Status	Description	Visible To
PENDING	Registration submitted, awaiting payment or confirmation	Organizer, Staff, Runner
CONFIRMED	Payment received and verified	Organizer, Staff, Runner
FAILED	Payment failed or expired	Organizer, Runner
REFUNDED	Payment returned to runner	Organizer, Runner
CANCELLED	Registration cancelled by runner or organizer	Organizer, Runner

Online Payment Workflow (PayMongo)

1. Runner completes registration form and selects GCash/Maya/Card
2. System creates PayMongo checkout session
3. Runner redirected to payment provider
4. On success, PayMongo sends webhook to TakboHub
5. System automatically marks registration as CONFIRMED

6. Confirmation email sent to runner

Bank Transfer Workflow

7. Runner completes registration form and selects "Bank Transfer"
8. System shows organizer's bank details and reference number
9. Runner transfers payment and uploads screenshot as proof
10. Registration saved as PENDING with proof attached
11. Organizer/Staff reviews proof in dashboard
12. Organizer/Staff clicks "Confirm Payment"
13. Confirmation email sent to runner

Target Users

Primary: Race Organizers (B2B)

- Local running clubs and communities
- LGU-organized fun runs and fiestas
- Schools and universities (intramurals, foundation runs)
- Corporate events and charity runs
- Professional race organizers managing multiple events

Secondary: Runners (B2C)

- Recreational runners looking for local events
- Competitive runners tracking race history
- Running groups and "barkadas" registering together

Solution Overview

TakboHub is a web-based platform (mobile-first, responsive) that provides:

14. **Organizer Dashboard:** Create and manage events, track registrations, handle payments, communicate with participants
15. **Public Event Pages:** Shareable registration pages with integrated payments
16. **Runner Experience:** Simple registration flow, payment via GCash/Maya/card/cash, instant confirmation

MVP Features (Phase 1)

Target: Launch with 1-2 pilot events within 4-6 weeks

Event Creation

- Event details: name, date, time, location (with map link), description
- Banner image upload
- Race categories (e.g., 3K, 5K, 10K, 21K)
- Pricing tiers (early bird, regular, late registration)
- Registration open/close dates
- Payment method configuration (enable/disable online, bank transfer, cash)

Registration Flow

- Runner info: full name, email, phone, birthdate, gender
- Race category selection
- Shirt size selection
- Emergency contact (name, phone, relationship)
- Waiver acknowledgment checkbox
- Payment method selection (based on organizer settings)

Payment Integration

- PayMongo integration (GCash, Maya, credit/debit cards)
- Bank transfer option with proof upload
- Cash payment option with manual confirmation
- Automatic payment confirmation (online) / manual confirmation (offline)
- Email receipt to runner
- Payment status tracking (pending, confirmed, failed, refunded)

Organizer Dashboard

- Registration list with search and filter (by status, category, payment method)
- Pending payments queue with proof-of-payment viewer
- One-click payment confirmation
- Export to CSV/Excel
- Summary stats: total registered, revenue, breakdown by category/shirt size/payment method
- Basic event settings management

Phase 2 Features (Post-MVP)

- Registration Staff role with limited permissions
- Walk-in registration mode (tablet/phone interface for race day)
- Bib number auto-assignment and management
- Race kit claiming tracker
- Bulk email/SMS communication (via Semaphore)
- Race results upload and publishing
- Finisher certificate generation
- Runner accounts and race history
- Event discovery feed
- Group registration
- Timing company integration API

Technical Architecture

TakboHub uses a modern, decoupled architecture with a React SPA frontend and NestJS REST API backend. This separation enables independent scaling, better team collaboration, and flexibility for future mobile app development.

Architecture Overview

Component	Technology	Responsibility
Frontend SPA	React + Vite	User interface, client-side routing, state management
Backend API	NestJS	Business logic, authentication, data persistence
Database	PostgreSQL	Primary data store, relational data
Cache/Queue	Redis	Session management, job queues, rate limiting
File Storage	MinIO (dev) / S3 (prod)	Event banners, payment proofs, exports

Frontend Stack

Layer	Technology	Version	Rationale
Build Tool	Vite	6.x	Fast HMR, optimized builds, modern ESM support
UI Framework	React	19.x	Component-based, large ecosystem, concurrent features
Routing	TanStack Router	1.x	Type-safe routing, file-based routes, search params
Server State	TanStack Query	5.x	Caching, background sync, optimistic updates
Client State	Zustand	5.x	Lightweight, simple API, devtools support
Styling	Tailwind CSS	4.x	Utility-first, JIT compilation, design system
Components	shadcn/ui	Latest	Accessible, customizable, copy-paste components
Forms	React Hook Form	7.x	Performance, validation integration, minimal re-renders
Validation	Zod	3.x	Type-safe schemas, shared with

			backend
HTTP Client	Axios	1.x	Interceptors, request/response transforms
Icons	Lucide React	Latest	Consistent icon set, tree-shakeable
Date Handling	date-fns	3.x	Modular, immutable, TypeScript support

Backend Stack

Layer	Technology	Version	Rationale
Framework	NestJS	11.x	Modular architecture, DI, enterprise patterns
Runtime	Node.js	22 LTS	Long-term support, performance improvements
Language	TypeScript	5.x	Type safety, better DX, maintainability
ORM	Prisma	6.x	Type-safe queries, migrations, studio GUI
Validation	class-validator	0.14.x	Decorator-based, NestJS native integration
Transform	class-transformer	0.5.x	DTO transformation, serialization
Auth	@nestjs/passport + JWT	Latest	Strategy-based auth, refresh tokens
API Docs	@nestjs/swagger	Latest	Auto-generated OpenAPI documentation
Config	@nestjs/config	Latest	Environment management, validation
Queue	BullMQ	5.x	Redis-backed job processing, retries, scheduling
Cache	@nestjs/cache-manager	Latest	Response caching, Redis integration
Rate Limit	@nestjs/throttler	Latest	Protect endpoints, configurable limits
File Upload	Multer	1.x	Multipart handling, storage engines
Logging	Pino + nestjs-pino	Latest	Structured JSON logs, high

			performance
Testing	Jest + Supertest	Latest	Unit tests, e2e tests, coverage

Database & Infrastructure

Component	Technology	Environment	Purpose
Primary DB	PostgreSQL	16.x	Relational data, ACID compliance, JSON support
Cache/Queue	Redis	7.x	Sessions, job queues, rate limiting, caching
File Storage	MinIO	Dev	S3-compatible local storage for development
File Storage	AWS S3 / DO Spaces	Prod	Scalable object storage for production
Containers	Docker + Compose	Latest	Consistent environments, easy setup
Reverse Proxy	Nginx	Prod	SSL termination, static files, load balancing

External Services

Service	Provider	Purpose	Phase
Payments	PayMongo	GCash, Maya, Cards, QR Ph processing	MVP
Email	Resend	Transactional emails (confirmations, receipts)	MVP
SMS	Semaphore	SMS notifications, OTP (Philippines-based)	Phase 2
Maps	Google Maps API	Event location display, directions	MVP

Project Structure

The project uses a monorepo structure with pnpm workspaces for shared code and dependencies:

```
takbohub/
├── docker-compose.yml          # Local development services
├── apps/
│   ├── web/                   # React + Vite frontend
│   │   ├── src/
│   │   │   ├── components/    # Reusable UI components
│   │   │   ├── features/      # Feature modules
│   │   │   ├── hooks/         # Custom React hooks
│   │   │   ├── lib/           # Utilities, API client
│   │   │   └── routes/         # TanStack Router pages
│   │   └── package.json
│   └── api/                   # NestJS backend
│       ├── src/
│       │   ├── modules/       # Feature modules
│       │   │   ├── auth/
│       │   │   ├── users/
│       │   │   ├── organizations/
│       │   │   ├── events/
│       │   │   ├── registrations/
│       │   │   ├── payments/
│       │   │   └── common/
│       │   ├── config/
│       │   └── main.ts
│       └── package.json
├── packages/
│   ├── shared/                # Shared code
│   │   ├── src/
│   │   │   ├── schemas/       # Zod validation schemas
│   │   │   └── types/          # TypeScript types
│   │   └── package.json
└── prisma/                    # Database schema & migrations
```

Core Data Models

The following Prisma schema defines the core entities and relationships:

Users

- id, email, name, role (SUPER_ADMIN, ORGANIZER, STAFF, RUNNER), passwordHash, emailVerified, createdAt, updatedAt

Organizations

- id, name, slug, ownerId, logoUrl, createdAt, updatedAt

OrganizationMembers

- id, organizationId, userId, role (ORGANIZER, STAFF), invitedAt, acceptedAt

Events

- id, organizationId, name, slug, description, date, location, bannerUrl
- registrationOpen, registrationClose, status (DRAFT, PUBLISHED, COMPLETED)
- paymentMethodsEnabled (JSON), bankDetails, cashPaymentInstructions

RaceCategories

- id, eventId, name, distance, price, earlyBirdPrice, earlyBirdDeadline, slotLimit, slotsTaken

Registrations

- id, eventId, categoryId, runnerName, email, phone, birthdate, gender
- shirtSize, emergencyContactName, emergencyContactPhone, emergencyContactRelationship
- paymentMethod (QRPH, GCASH, MAYA, CARD, BANK_TRANSFER, CASH, FREE)
- paymentStatus (PENDING, CONFIRMED, FAILED, REFUNDED, CANCELLED)
- proofOfPaymentUrl, amountPaid, registeredAt, bibNumber, confirmedBy, confirmedAt, confirmationNotes

Payments

- id, registrationId, paymongoPaymentId, paymongoCheckoutId, amount, status, paymentMethod, paidAt, metadata (JSON)

API Design

The REST API follows standard conventions with versioning and consistent response formats:

Authentication Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/register	Register new user
POST	/api/v1/auth/login	Login and receive tokens
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/auth/logout	Invalidate refresh token
POST	/api/v1/auth/forgot-password	Request password reset
POST	/api/v1/auth/reset-password	Reset password with token

Event Endpoints

Method	Endpoint	Description
GET	/api/v1/events	List public events
GET	/api/v1/events/:slug	Get event by slug (public)
POST	/api/v1/organizations/:orgId/events	Create event
PATCH	/api/v1/events/:id	Update event
DELETE	/api/v1/events/:id	Delete event
POST	/api/v1/events/:id/publish	Publish event

Registration Endpoints

Method	Endpoint	Description
POST	/api/v1/events/:eventId/register	Create registration
GET	/api/v1/events/:eventId/registrations	List registrations (organizer)
GET	/api/v1/registrations/:id	Get registration details
PATCH	/api/v1/registrations/:id/confirm	Confirm payment
POST	/api/v1/registrations/:id/cancel	Cancel registration

Payment Endpoints

Method	Endpoint	Description
POST	/api/v1/payments/checkout	Create PayMongo checkout
POST	/api/v1/payments/webhook	PayMongo webhook handler
POST	/api/v1/registrations/:id/upload-proof	Upload payment proof

Business Model

Revenue Streams

Model	Structure	Example
Transaction Fee	3-5% per registration	₱500 reg × 500 runners × 5% = ₱12,500/event
Flat Fee	₱2,000-₱10,000 per event	Based on expected participants
Freemium	Free basic, paid features	SMS, certificates, priority support
Featured Listing	Pay for visibility	₱1,000-₱5,000 for homepage feature

MVP Pricing Strategy

- **First 2-3 events:** FREE (for testimonials and feedback)
- **Initial pricing:** 3% transaction fee OR ₱2,000 flat fee for events under 200 runners
- **Re-evaluate:** After 10 events based on organizer feedback

Success Metrics

MVP Success Criteria (3 months)

- 5+ events successfully completed on platform
- 500+ total registrations processed
- 90%+ payment success rate (online payments)
- < 24 hour average time to confirm offline payments
- At least 2 organizers willing to pay for next event
- Net Promoter Score (NPS) > 30 from organizers

Key Performance Indicators

- Registration conversion rate (visitors → registrations)
- Payment success rate (by method)
- Average time to complete registration
- Average time to confirm offline payments
- Organizer retention rate (% who use for subsequent events)
- Support ticket volume per event

Development Timeline

Phase	Duration	Deliverables
Week 1-2	Foundation	Monorepo setup, Docker environment, database schema, auth module, basic UI components
Week 3-4	Core Features	Event creation, registration flow, PayMongo integration, offline payment support
Week 5	Dashboard	Organizer dashboard, participant list, payment confirmation UI, exports
Week 6	Polish & Launch	Testing, bug fixes, pilot event deployment

Risks and Mitigations

Risk	Impact	Mitigation
Payment failures on event day	High - lost registrations	Thorough PayMongo testing, cash/bank fallback options
Organizers slow to confirm offline payments	Medium - poor runner experience	Dashboard notifications, email reminders, pending payment alerts
Organizers resistant to change	Medium - slow adoption	Free pilot, white-glove onboarding, show ROI
Competition from larger platforms	Medium	Focus on local market, Filipino UX, relationships
Technical issues during event	High - reputation damage	Load testing, on-call support for pilots
Low runner trust in new platform	Medium	Organizer branding, clear payment confirmation, familiar payment options

Competitive Landscape

Competitor	Strengths	Weaknesses	Our Advantage
Google Forms + GCash	Free, familiar	Manual, no payment integration	Automated, integrated
Race Roster	Full-featured, used by RUNRIO	Expensive (~7%), overkill for small events	Cheaper, simpler, local support
Eventbrite	Full-featured, trusted	No GCash/Maya, not PH-focused	Local payments, localized
Ticket2Me	PH-based, multiple event types	Not running-specific	Purpose-built for runs

Go-to-Market Strategy

Phase 1: Local Validation (Dumaguete/Negros Oriental)

17. Identify 2-3 upcoming local events (LGU runs, university events, club runs)
18. Offer free platform usage in exchange for feedback and testimonial
19. Provide white-glove support during events
20. Document case studies with real metrics

Phase 2: Regional Expansion (Visayas)

- Leverage testimonials to approach Cebu, Bacolod, Iloilo organizers
- Partner with regional running clubs and groups
- Attend/sponsor local running events for visibility

Phase 3: National Scale

- Approach established race organizers
- Launch runner-facing discovery features
- Consider mobile app development

Appendix

User Stories

As an organizer, I want to...

- Create an event with multiple race categories so runners can choose their distance
- Enable cash payments so runners without e-wallets can still register
- See a list of pending payments so I can follow up or confirm them
- Confirm a payment with one click after verifying cash/bank transfer
- See real-time registration counts so I know how the event is selling
- Export participant data so I can prepare race kits and bibs
- Know immediately when someone pays online so I don't have to check manually

As a registration staff member, I want to...

- View the list of registrations for an event I'm assigned to
- Confirm a cash payment when a runner pays at the booth
- Add a walk-in registration on race day (Phase 2)

As a runner, I want to...

- Register and pay in under 3 minutes so I can secure my slot quickly
- Pay with GCash because that's what I use for everything
- Choose to pay with cash if I prefer to pay in person
- Upload proof of my bank transfer so the organizer can verify it
- Get instant confirmation so I know my registration went through
- See all the event details on one page so I know what I'm signing up for

— End of Document —